

Comprendre ton projet. Le présenter avec des preuves.

Un seul conducteur de 15 minutes, les explications de chaque brique, les gestes de démonstration et les réponses aux questions de Jérémy.

État vérifié aujourd'hui

76 tests passent ; Docker et Swagger fonctionnent. La démo directe a été rétablie avec minstral-8b-2512. Les scores de juillet concernent l'ancien modèle. La démo de secours est explicitement enregistrée. Le corpus couvre juillet 2025 → juillet 2026 ; il ne représente pas un agenda actualisé pour aujourd'hui.

COMPRENDRE AVANT DE RÉCITER

Ce que tu as construit, en une minute

Un programme récupère des fiches d'événements, les transforme en passages et en vecteurs, retrouve les passages utiles pour une question, puis demande à Mistral de rédiger une réponse. Une API permet aux autres logiciels de l'utiliser.

Tu n'as pas créé Mistral. Tu as intégré des modèles existants, préparé les données, construit l'index et écrit le code qui relie recherche, réponse et API. Le but était de démontrer une chaîne utilisable par les équipes produit et marketing de Puls-Events.

L'analogie à retenir

Les fiches OpenAgenda sont le catalogue. Les chunks sont les passages de ce catalogue. Les embeddings servent à comparer leur sens. FAISS est le bibliothécaire qui trouve les passages. Le prompt donne les consignes et les passages à Mistral. Mistral rédige. FastAPI est le guichet pour poser la question. Docker permet de transporter le même environnement.

Les deux moments du projet

1. **Avant les questions** : collecte → nettoyage → Parquet → découpage → embeddings Mistral → sauvegarde FAISS. Cette préparation appelle déjà internet.
2. **À chaque question** : embedding Mistral de la question → recherche locale de quatre passages → contexte + prompt → chat Mistral → JSON avec réponse et sources.

Les chiffres qui correspondent vraiment au code

À mémoriser	Ce que cela veut dire
7 586 événements	7 586 identifiants uniques dans le corpus livré, tous représentés dans les chunks.
14 903 passages	Nombre de chunks de tout le corpus, égal au nombre de vecteurs FAISS.
1 000 / 200 caractères	Taille de découpage / chevauchement maximal. Ce ne sont pas des tokens.
1 024 dimensions	1 024 nombres par vecteur, vérifiés dans index.faiss.
top-k = 4	Quatre passages recherchés ; potentiellement moins de quatre événements après déduplication des sources.
76 tests	Suite passée le 1er octobre : 72 passed in 3.06s, sans preuve de disponibilité Mistral.
4 cas locaux / 3 cas RAGAS	Deux jeux distincts et deux méthodes distinctes.
0,900 / 0,846 / 1,000 / 0,683	Fidélité / pertinence / précision du contexte / correction, sur trois cas le 23 juillet.

Les six étapes demandées, et où tu les retrouves

Étape	Ce que fait le code	Fichiers à connaître
1 · Environnement	Python 3.12, versions directes des dépendances, variables d'environnement.	environment.yml requirements.txt scripts/check_env.py
2 · Collecte	Filtre ville/date, parcourt les pages, nettoie HTML et champs, exporte JSON/Parquet/manifeste.	scripts/fetch_events.py src/openagenda_rag/ingestion.py
3 · Index	Découpe, calcule les embeddings par lots, construit et persiste FAISS avec métadonnées.	scripts/build_index.py src/openagenda_rag/indexing.py
4 · Chatbot	Retrouve les chunks, compose le prompt, appelle le chat et sérialise la réponse.	scripts/chatbot.py src/openagenda_rag/rag.py src/openagenda_rag/service.py
5 · API	Valide le JSON, appelle le service, expose les routes et Swagger.	scripts/run_api.py src/openagenda_rag/api.p

Étape	Ce que fait le code	Fichiers à connaître
		y scripts/api_test.py
6 · Livraison	Image, volume, seed et lancement local ; scripts de démonstration et supports.	Dockerfile docker-compose.yml scripts/docker_entrypoint.sh scripts/demo_api_5min.py
Validation transversale	Tests du code, cas annotés, règles locales et quatre métriques RAGAS.	tests/ scripts/evaluate_rag.py scripts/evaluate_ragas.py

Un événement réel de bout en bout

Concert Fishers est la fiche 57857032. Le texte contient notamment « reprises pop, rock, funk ». Les métadonnées donnent Paris, Le Gymnase Montparnasse, le 21 juin 2026 de 19 h 30 à 23 h 30, et l'URL OpenAgenda. Le chunk 57857032 : : chunk-0 possède un vecteur. La question « Parle-moi de Concert Fishers a Paris » permet de le retrouver et de fournir ces faits au modèle. Ce concert est passé au jour de la soutenance ; il sert à vérifier la chaîne sur le corpus livré.

Le parcours pour te préparer

1. **15 min** : lis « Comprendre » et le glossaire ; explique à voix haute la différence embedding / FAISS / chat.
2. **25 min** : lis le script des 12 slides dans l'ordre, avec les explications simples. Ne cherche pas à apprendre tous les détails techniques.
3. **15 min** : répète la démo avec la page dédiée ; apprends à annoncer le mode enregistré et à montrer les sources.
4. **15 min** : lis les réponses sur le corpus, top-k, les scores et les limites. Réponds d'abord en une phrase.
5. **15 min** : lance le chronomètre du deck et fais une répétition complète. Les durées incluent les changements d'écran, la démo et le dépôt.
6. **Avant 14 h** : vérifie son, partage de fenêtre, liens, Docker et mode secours. Garde de la marge pour te connecter.

Démo directe rétablie : modèle à annoncer

Mistral Small renvoyait 429 alors que les embeddings fonctionnaient. La génération a été basculée sur ministral-8b-2512 et les deux scénarios ont réussi en direct. Annonce ce changement ; les scores RAGAS de juillet restent ceux de l'ancien modèle. Compare toujours la sortie aux sources.

Écran partagé et écran personnel

Partage la **fenêtre** du diaporama ou de la démo. Ce guide contient tes notes et doit rester hors partage. Si tu n'as qu'un écran, place deux fenêtres côte à côte et partage la bonne fenêtre. Évite de partager tout le bureau. Utilise un second écran ou ce guide imprimé si disponible.

Le contrat de la soutenance

15 minutes de livrables, 10 minutes de discussion, 5 minutes de débrief. Le jury joue Jérémy pendant les deux premières parties. La soutenance porte sur la mission Puls-Events, pas sur les exercices municipaux du cours également présents dans le document exporté. Cible 15 minutes ; en dessous de 10 ou au-dessus de 20 minutes, la présentation peut être refusée.

Ce qui est prêt, ce qui doit rester explicite

Vérifié aujourd'hui

76 tests passent. Docker, /health et Swagger répondent. Les deux scénarios /ask ont réussi avec ministral-8b-2512. Les volumes et dimensions de FAISS sont vérifiés.

Preuves historiques

Réponses de démo enregistrées ; RAGAS du 23 juillet. Le corpus a été collecté le 10 juillet. Les deux scénarios ont aussi réussi en direct aujourd'hui avec Ministral 8B ; RAGAS n'a pas été recalculé.

La question « explique-moi ton projet »

« J'ai construit un prototype de recommandation culturelle pour Puls-Events. J'ai préparé des données OpenAgenda de Paris, découpé les fiches, calculé leurs embeddings et enregistré les vecteurs dans FAISS. Pour une question, je récupère quatre passages et les fournis à Mistral pour générer une réponse sourcée. J'expose ce moteur via FastAPI et Docker. Les tests valident le code, et une évaluation sur un petit jeu annoté identifie les points à améliorer. Le corpus et les garde-fous doivent être renforcés avant production. »

Les mots que tu dois pouvoir expliquer

POC

Prototype qui démontre la faisabilité, avec un périmètre limité.

LLM

Modèle de langage qui prédit et rédige du texte ; ici Mistral pour la réponse.

RAG

Recherche de documents puis génération enrichie par ces documents.

Chunk

Passage de texte découpé pour la recherche.

Embedding / vecteur

Liste de nombres qui représente un texte pour comparer sa proximité avec d'autres textes.

Dimension

Nombre de coordonnées dans le vecteur : 1 024 dans l'index livré.

FAISS

Bibliothèque de recherche de voisins parmi les vecteurs ; elle ne rédige pas.

IndexFlatL2

Index exact qui compare les distances euclidiennes au carré ; distance faible = vecteurs proches.

Retriever / top-k

Sélecteur des k passages les plus proches ; k vaut 4 ici.

Contexte / prompt

Le contexte contient les passages ; le prompt contient aussi la question et les consignes de réponse.

Métadonnées

Informations autour du texte : identifiant, ville, lieu, dates, URL...

LangChain

Briques Python pour manipuler documents, découpage, recherche et appels aux modèles.

API REST / endpoint

Contrat HTTP qui permet à un logiciel d'appeler une route comme POST /ask.

JSON

Format des messages de l'API : champs question, answer, sources...

Parquet

Format de tableau compact utilisé pour les données normalisées.

Seed / manifeste

Le seed est le corpus livré ; le manifeste trace sa source, ses paramètres et ses volumes.

Docker / volume

Environnement conteneurisé ; le volume garde les fichiers entre deux créations du conteneur.

Température / token

Température : réglage de variation du texte. Token : unité de texte du modèle, différente d'un caractère.

RAGAS

Bibliothèque d'évaluation du contexte et de la réponse, avec des modèles juges.

Doublure / mock

Service simulé pour tester le code sans appels externes.

CI/CD

Automatisation des validations et déploiements ; elle est proposée, pas livrée ici.

Le conducteur unique

Les durées totalisent 15 minutes, avec la démo dans la slide 8 et le parcours rapport/dépôt dans la slide 12. Le texte est un conducteur : parle posément, montre les preuves et marque les transitions. Une lecture sans gestes sera plus courte ; répète avec le chronomètre.

Slide	Rôle	Repère	Durée
01	La mission Puls-Events	00:00 → 00:45	45 s
02	Besoin métier et principe	00:45 → 01:50	65 s
03	Données et préparation	01:50 → 03:05	75 s
04	Chunking et embeddings	03:05 → 04:20	75 s
05	Recherche et génération	04:20 → 05:30	70 s
06	Architecture du projet	05:30 → 06:30	60 s
07	FastAPI et Docker	06:30 → 07:35	65 s
08	Démonstration API	07:35 → 09:55	140 s
09	Validation et tests	09:55 → 11:00	65 s
10	Évaluation des réponses	11:00 → 12:40	100 s
11	Limites et risques observés	12:40 → 13:45	65 s
12	Livraison et perspectives	13:45 → 15:00	75 s

Après cela : 10 minutes de discussion, puis 5 minutes de débrief. Ne lis pas les explications techniques pendant les slides ; elles sont là pour comprendre et répondre.

Un assistant culturel fondé sur des événements réels

L'idée à retenir : Un POC complet pour tester la recherche en langage naturel.

Bonjour Jérémy. Je vais vous présenter le prototype réalisé pour Puls-Events, un assistant de recommandation culturelle qui utilise les événements OpenAgenda. Le besoin est de permettre aux utilisateurs de poser une question naturellement, puis de recevoir une réponse qu'ils peuvent vérifier grâce aux sources. Mon livrable couvre toute la chaîne : collecte et préparation des données, recherche vectorielle, génération de réponse, API et conteneur Docker. Le corpus livré contient 7 586 événements parisiens. Il s'agit d'un POC, donc d'une preuve de faisabilité. Je vais expliquer les choix, montrer l'API et présenter les résultats ainsi que les limites avant un déploiement plus large.

À montrer / à faire

Partage uniquement la fenêtre du diaporama. Garde ce guide hors du partage.

▼ Comprendre simplement

Tu joues le rôle du freelance qui livre un prototype à son client. Jérémy est le responsable technique. Tu présentes une solution testable, avec ses preuves et ses limites. POC signifie preuve de concept : vérifier que l'idée fonctionne avant de construire un produit à grande échelle.

▼ Si le jury creuse la technique

Le brief demande LangChain, Mistral, FAISS, une API, les scripts de reconstruction, des tests, un jeu annoté, un rapport, Docker et une démo. L'historique de conversation n'est pas nécessaire. Le modèle n'a pas été entraîné ou ajusté dans ce projet.

▼ La preuve dans le projet

README.md ; document de mission, message de Jérémy ; seed-data/processed/fetch_manifest.json.

Le piège à éviter

Ne dis pas que c'est déjà un produit en production ni que tu as entraîné Mistral.

Transition à dire : « Commençons par le risque métier que le RAG cherche à réduire. »

Le RAG relie la réponse au catalogue

L'idée à retenir : Rechercher les informations, puis rédiger à partir de ces informations.

Le problème métier est qu'une recherche par mots-clés ne correspond pas toujours à la façon dont un utilisateur exprime son envie. Par exemple, une sortie en famille peut correspondre à une visite jouée par des comédiens, même si les mots utilisés diffèrent. Un modèle de langage seul peut produire une réponse convaincante sans connaître les événements du catalogue. Le RAG combine deux capacités. Il recherche d'abord des passages proches du sens de la question. Il ajoute ensuite ces passages au contexte du modèle. Enfin, Mistral formule une réponse en français. Les sources permettent de revenir aux événements d'origine. Le RAG réduit ainsi le risque d'invention, mais ne le supprime pas : il faut encore vérifier la pertinence des passages et la fidélité de la réponse.

À montrer / à faire

Pointe les trois étapes. L'exemple famille existe dans le jeu RAGAS ; ce n'est pas une nouvelle preuve live.

▼ Comprendre simplement

Imagine un conseiller avec un catalogue devant lui. FAISS trouve les fiches utiles ; Mistral les lit et rédige. Retrieval = recherche. Augmented = on ajoute les fiches à la question. Generation = le modèle rédige. Le catalogue reste à l'extérieur du modèle.

▼ Si le jury creuse la technique

Ce n'est ni du fine-tuning ni de l'entraînement. L'embedding représente le texte numériquement, le retriever sélectionne des documents et ChatPromptTemplate assemble les messages transmis au chat. Les documents sont consultés à chaque question.

▼ La preuve dans le projet

src/openagenda_rag/rag.py : answer_question(), build_prompt() ; tests/fixtures/ragas_eval_dataset.csv.

Le piège à éviter

Ne promets pas zéro hallucination. Une recherche proche n'est pas nécessairement une réponse pertinente.

Transition à dire : « Cette approche dépend d'abord de la qualité du catalogue préparé. »

Le corpus est un instantané explicite de Paris

L'idée à retenir : Paris · du 10 juillet 2025 au 10 juillet 2026 · 7 586 événements.

Les données proviennent d'OpenAgenda à travers le jeu public exposé par Opendatasoft. Cette source est celle du lien donné dans la mission et ne demande pas de clé pour la collecte. Le script filtre Paris et la première date des événements, du 10 juillet 2025 au 10 juillet 2026. Il parcourt les résultats, nettoie les descriptions HTML, gère les champs absents et supprime les doublons par identifiant. Il produit un fichier Parquet structuré et un manifeste qui trace les paramètres et le volume. Je conserve les dates, le lieu et l'URL pour que la réponse soit vérifiable. Cette fenêtre était celle du POC construit en juillet. Pour une utilisation aujourd'hui, le corpus doit être actualisé : je ne présente pas ce catalogue figé comme un agenda à jour.

À montrer / à faire

Explique la fenêtre telle qu'elle est. Si on te demande le fichier, ouvre le manifeste de collecte, pas le JSON brut entier.

▼ Comprendre simplement

Une API de données donne des fiches d'événements. Le nettoyage rend leurs champs utilisables. Parquet est un format de tableau compact. Le manifeste est la fiche d'identité de la collecte : source, filtres, date et quantité. Le seed est la copie de référence livrée avec le projet.

▼ Si le jury creuse la technique

`build_records_where_clause()` filtre `firstdate_begin`, pas chaque occurrence récurrente. `last_timing` peut dépasser juillet 2026. La borne haute fixe exclut les événements qui commencent après la collecte. Les données ne sont pas automatiquement rafraîchies. 39 titres sont absents dans le seed, mais leurs descriptions peuvent rester indexables. Déduplication par `event_uid` ; les titres identiques avec des UID différents ne sont pas fusionnés.

▼ La preuve dans le projet

`scripts/fetch_events.py` ; `src/openagenda_rag/ingestion.py` ; `seed-data/processed/fetch_manifest.json` ; `events.parquet`.

Le piège à éviter

La consigne demande des événements récents et vise des recommandations à venir. Au 1er octobre, certains événements du seed ont plus d'un an. La fraîcheur constitue donc un écart actuel à signaler et à corriger par une nouvelle collecte, pas à masquer.

Transition à dire : « Une fois les fiches préparées, il faut les rendre recherchables par leur sens. »

03:05 → 04:20 • 75 secondes • repère incluant gestes et pauses

Les descriptions deviennent des passages recherchables

L'idée à retenir : 1 000 caractères par passage, 200 de chevauchement, vecteurs de 1 024 dimensions.

Certaines fiches sont longues. Les indexer comme un seul bloc risquerait de diluer une information utile. J'utilise donc le découpage récursif de LangChain, configuré à 1 000 caractères avec un chevauchement maximal de 200 caractères. Le chevauchement limite la perte d'information aux frontières. Par exemple, une fiche CHIMERE contient 10 273 caractères et produit 15 passages, car le découpage tient aussi compte des séparateurs. Le modèle mistral-embed transforme chaque passage en un vecteur de 1 024 dimensions, autrement dit une liste de nombres représentant son sens. La construction se fait par lots de 50. Au total, FAISS contient 14 903 vecteurs pour les 7 586 événements. Chaque passage conserve l'identifiant de l'événement, ses dates, son lieu et son lien.

À montrer / à faire

Distingue la fiche d'exemple et les volumes de tout le corpus. Aucun réindexage en direct.

▼ Comprendre simplement

Un chunk est un morceau de texte. Un embedding est une liste de nombres qui permet de comparer le sens des textes. FAISS est l'outil qui range ces vecteurs et retrouve les plus proches. Il ne rédige aucune phrase et ne remplace pas Mistral.

▼ Si le jury creuse la technique

RecursiveCharacterTextSplitter, add_start_index=True. Le chevauchement est un paramètre maximal, pas 200 caractères garantis à chaque frontière. chunk_id = event_uid::chunk-N. L'index réel est IndexFlatL2 : recherche exacte selon la distance euclidienne au carré, sans normalisation explicite dans le code. Moins la distance est élevée, plus les vecteurs sont proches. Tous les 7 586 UID sont représentés. CHIMERE 69427795 a 15 chunks ; une autre fiche porte le même titre avec un UID différent.

▼ La preuve dans le projet

src/openagenda_rag/indexing.py : build_documents(), build_vector_store() ; seed-data/index/index_manifest.json ; index.faiss.

Le piège à éviter

Caractères et tokens sont différents. Ne dis pas que CHIMERE seul produit 14 903 chunks. Un vecteur n'est ni un résumé ni une réponse.

Transition à dire : « L'index est préparé. Voyons ce qui se passe lorsqu'un utilisateur pose une question. »

Mistral reçoit la question et quatre passages

L'idée à retenir : L'embedding retrouve ; le modèle de chat rédige.

Au moment d'une question, on utilise le même modèle d'embedding que pour les documents. Cela rend leurs vecteurs comparables. FAISS sélectionne les quatre passages les plus proches : c'est le paramètre top-k. Le code assemble ensuite un contexte avec le texte et les métadonnées, puis le transmet avec la question à minstral-8b-2512. Pour la soutenance, ce modèle remplace mistral-small-latest, limité par Mistral aujourd'hui. Le prompt demande de répondre seulement à partir du contexte, de citer les informations disponibles et de ne pas inventer. La température est basse, à 0,1, pour limiter la variation ; la réponse est limitée à 700 tokens. Le JSON renvoie la réponse et les sources récupérées, regroupées par événement. Attention : quatre passages peuvent provenir du même événement. Et l'absence de seuil de pertinence signifie que le modèle doit encore décider si le contexte permet de répondre.

À montrer / à faire

Pointe la séparation embedding/chat et le texte du prompt. Si demandé, ouvre SYSTEM_PROMPT dans rag.py.

▼ Comprendre simplement

Les deux modèles ont deux métiers différents. mistral-embed transforme textes et questions en nombres. ministral-8b-2512 lit le contexte et produit des phrases. top-k signifie combien de passages on lui donne ; cela ne signifie pas quatre recommandations.

▼ Si le jury creuse la technique

answer_question() appelle retriever.invoke(), build_source_entries(), puis prompt.invoke() et chat_model.invoke(). Les sources sont les événements récupérés, pas une validation que chaque source soutient chaque phrase. Aucun filtre de date ou de ville n'est appliqué dynamiquement dans le retriever. L'absence de documents déclenche un refus déterministe ; en recherche top-k, le résultat non vide est fréquent même hors sujet. random_seed=42 stabilise sans garantir une sortie identique. L'alias latest peut évoluer. Le modèle de démonstration est fixé à ministral-8b-2512 dans .env. L'embedding reste mistral-embed : l'index est compatible sans reconstruction. Les scores RAGAS historiques concernent mistral-small-latest et ne valident pas ce changement. La date du jour et un statut temporel calculé sont transmis au prompt ; aucun filtre de date ne supprime les passages.

▼ La preuve dans le projet

src/openagenda_rag/rag.py ; src/openagenda_rag/settings.py : load_rag_settings().

Le piège à éviter

Ne dis pas que les sources renvoyées sont toutes utilisées dans la réponse ni que le refus Lyon est codé comme une règle de ville.

Transition à dire : « Ces étapes sont séparées dans le code pour pouvoir les tester et les remplacer. »

La préparation de l'index précède les questions

L'idée à retenir : Deux parcours distincts : construire le catalogue et répondre aux utilisateurs.

L'architecture distingue la préparation des données du traitement des questions. La préparation collecte les fiches, les normalise, calcule leurs embeddings et sauvegarde l'index. Elle est faite avant l'utilisation du chatbot et appelle déjà Mistral pour les vecteurs. À chaque question, l'API appelle le service RAG, qui recharge l'index si nécessaire, recherche les passages et sollicite le modèle de chat. Le retriever et le client Mistral sont initialisés à la première question puis conservés en mémoire pour les suivantes. Le dossier scripts contient les commandes de lancement, tandis que src contient la logique métier. Cette séparation permet d'utiliser le même moteur depuis le terminal, l'API et l'évaluation. Elle évite de relancer la collecte ou l'indexation pour chaque demande.

À montrer / à faire

Si Jérémy demande le code, montre service.py puis rag.py. Évite de naviguer entre toutes les fonctions.

▼ Comprendre simplement

La préparation construit une bibliothèque. L'usage consulte cette bibliothèque. Le service est le point central qui relie les briques. LangChain fournit les objets pour les documents, le découpage, la recherche et les messages ; il ne remplace pas le code du projet.

▼ Si le jury creuse la technique

OpenAgendaRAGService._ensure_runtime() initialise les objets paresseusement. Ce sont des clients et un retriever en mémoire, pas les poids de Mistral en local. rebuild() reconstruit depuis le Parquet et invalide ces objets ; il ne recollecte pas les événements. La CLI réutilise le service ; la génération reste distante même sans HTTP local.

▼ La preuve dans le projet

src/openagenda_rag/service.py ; scripts/chatbot.py ; scripts/run_api.py.

Le piège à éviter

Ne qualifie pas la préparation de complètement hors ligne : elle appelle Mistral. /health ne charge pas l'index et ne teste pas la clé.

Transition à dire : « L'API donne aux équipes un contrat simple pour tester cette chaîne. »

L'API rend le POC accessible aux équipes

L'idée à retenir : Une question HTTP, une réponse JSON, un environnement conteneurisé.

FastAPI expose trois routes métier. Health indique que le serveur répond et montre sa configuration. Ask accepte une question et renvoie un JSON avec la réponse, les sources et le nombre de passages récupérés. Rebuild reconstruit l'index depuis le fichier Parquet ; il ne lance pas une nouvelle collecte. Swagger, sur docs, permet de lire et tester le contrat sans écrire de client. Docker embarque le code, les dépendances et le seed. Au premier démarrage, l'index et les données sont copiés dans un volume persistant. La clé Mistral est injectée par l'environnement et n'est pas versionnée. Le jeton d'administration de rebuild est optionnel et doit être configuré avant une exposition publique. Le POC reste dépendant d'internet pour vectoriser la question et générer la réponse.

À montrer / à faire

Pendant la démo, ouvre Swagger et montre les routes ; garde /rebuild fermé. Tu peux cliquer le lien Démo de la slide suivante.

▼ Comprendre simplement

Une API est une porte d'entrée pour d'autres logiciels. GET lit, POST envoie une action ou des données. JSON est le format des messages. Swagger est la page de documentation interactive. Docker fournit le même environnement ; le volume garde les fichiers modifiés lorsque le conteneur est recréé.

▼ Si le jury creuse la technique

Validation Pydantic : question vide → 422 ; espaces seuls → 400 ; index absent → 503. Les erreurs HTTP Mistral 429 et d'accès sont contrôlées en 503, les autres erreurs HTTP fournisseur en 502, les erreurs réseau en 503. Les détails des exceptions fournisseur et inattendues ne sont plus exposés. /rebuild est protégé uniquement si API_REBUILD_TOKEN est défini. /health vérifie le serveur et sa configuration, sans appeler Mistral.

▼ La preuve dans le projet

src/openagenda_rag/api.py ; Dockerfile ; docker-compose.yml ; scripts/docker_entrypoint.sh.

Le piège à éviter

Ne dis pas que /rebuild est forcément protégé ou que /health valide Mistral et l'index.

Transition à dire : « Je vais maintenant montrer un cas présent dans le corpus et un cas hors périmètre. »

La démonstration montre un résultat et une limite

L'idée à retenir : Un événement parisien sourcé, puis une demande à Lyon hors corpus.

Je commence par montrer que l'API répond et que ses routes sont documentées. Pour le premier cas, je demande des informations sur Concert Fishers à Paris. Le résultat doit permettre de retrouver l'événement, son lieu, sa date et son URL. Il s'agit d'un exemple du corpus livré, pas d'une sortie à venir aujourd'hui. Je compare la réponse au premier événement source. Les autres sources sont les voisins récupérés et ne sont pas toutes nécessaires à cette réponse. Pour le second cas, je demande des expositions photo à Lyon. Le corpus contient Paris. Dans la répétition directe d'aujourd'hui, le modèle dit qu'il ne sait pas et n'invente pas d'exposition lyonnaise. FAISS renvoie quand même des passages : cela illustre la limite de la recherche sans seuil de pertinence.

À montrer / à faire

140 s au total : 20 s health/Swagger ; 50 s cas nominal ; 40 s Lyon ; 30 s explication et retour au deck. Une tentative live maximum en cas de quota, puis enregistrement. Mode secours disponible hors réseau.

▼ Comprendre simplement

Ta démo doit prouver trois choses : le contrat API existe ; la réponse utilise une vraie fiche ; le système sait reconnaître au moins un cas hors corpus. Le premier événement est ancien : annonce-le. Si Mistral est indisponible, montre les réponses enregistrées en le disant explicitement.

▼ Si le jury creuse la technique

Le 1er octobre, les deux scénarios /ask ont réussi en direct avec minstral-8b-2512. Le rapport outputs/demo/demo_api_2026-10-01.json en conserve les réponses, le modèle et l'horodatage. La mesure couvre health et les deux questions, pas chaque requête ni un benchmark. Mistral Small renvoyait 429 ; le modèle de génération a été changé, sans modifier mistral-embed ni reconstruire FAISS. La sortie peut encore contenir un conseil général non sourcé : le signaler si présent.

▼ La preuve dans le projet

scripts/demo_api_5min.py ; outputs/demo/demo_api_2026-10-01.json ; démonstrateur HTML ; état de préparation.

Le piège à éviter

Ne présente pas un enregistrement comme un appel direct. Compare les faits et les dates aux sources. Si un conseil général non sourcé apparaît, explique que le prompt réduit les erreurs sans les éliminer. Aucun secret à l'écran.

Transition à dire : « La démonstration donne un exemple ; les tests et les métriques complètent cette preuve. »

09:55 → 11:00 • 65 secondes • repère incluant gestes et pauses

Les tests du code et la qualité des réponses se complètent

L'idée à retenir : 76 tests réussis aujourd'hui ; quatre cas fonctionnels annotés.

J'ai exécuté aujourd'hui la suite : les 76 tests passent. Ils vérifient notamment les filtres, la normalisation, la construction et le rechargement de l'index, la forme des réponses, la validation API et le calcul des métriques. Ces tests utilisent des doublures pour les services externes. Ils protègent les comportements du code, sans garantir que Mistral est accessible ni que toutes les réponses métier sont bonnes. Un deuxième niveau contient quatre questions annotées avec des réponses et des titres attendus. Les résultats enregistrés classent les quatre cas comme corrects selon les règles locales. Ces règles sont assez permissives : un titre peut être retrouvé dans les sources sans que la réponse soit parfaite. Je complète donc cette évaluation par RAGAS. L'automatisation en CI serait une prochaine étape ; elle n'est pas implémentée ici.

À montrer / à faire

Annonce le résultat des tests, puis explique ce qu'ils couvrent. Montre le petit jeu CSV si demandé.

▼ Comprendre simplement

Les tests de code vérifient les mécanismes. Le jeu annoté vérifie des exemples de questions. RAGAS examine la qualité du texte et du contexte. Ce sont trois niveaux de preuve différents. Une doublure simule un service externe pour tester sans internet et sans coût.

▼ Si le jury creuse la technique

evaluation.py utilise SequenceMatcher sur du texte normalisé : similarité lexicale, pas embedding sémantique. exact_match_soft = égalité après normalisation. matched_expected_titles cherche dans answer OU sources. Sans titre attendu, certaines expressions de refus suffisent à classer correct. Les sorties historiques ont 4 corrects, exact_match_soft_rate=0 et similarité moyenne ~0,131. Ce n'est pas contradictoire, car les mesures ne testent pas la même chose. --ragas sur evaluate_rag.py ne lance pas les quatre métriques ; le vrai script est evaluate_ragas.py.

▼ La preuve dans le projet

tests/ ; tests/fixtures/rag_eval_dataset.csv ; scripts/evaluate_rag.py ; outputs/evaluation/latest_summary.json.

Le piège à éviter

Ne transforme pas 4/4 en une précision de 100 % du système. Les tests ne font pas tous des appels Mistral réels.

Transition à dire : « RAGAS nous aide à distinguer une bonne recherche d'une réponse pleinement correcte. »

Les scores RAGAS orientent les améliorations

L'idée à retenir : Trois cas évalués : fidélité élevée, correction encore à améliorer.

L'évaluation RAGAS porte sur trois cas : Concert Fishers, le Salon de la Photo et une sortie en famille avec visite théâtralisée. Pour chaque cas, je conserve la question, les passages réellement transmis au modèle, la réponse et une référence humaine. Un modèle juge calcule quatre métriques. La fidélité mesure si la réponse est soutenue par le contexte. La pertinence mesure son adéquation à la question. La précision du contexte apprécie la pertinence et le classement des passages récupérés ; elle ne mesure pas la couverture de tout le catalogue. La correction compare la réponse à la référence. Les moyennes enregistrées le 23 juillet sont 0,900, 0,846, 1,000 et 0,683. Le dernier score montre que des écarts demeurent même avec un contexte utile. Sur le cas famille, le score de correction est d'environ 0,581 et la réponse ajoute des informations pratiques à vérifier. La priorité est donc de mieux cadrer la réponse, contrôler les faits et élargir le jeu d'évaluation. Trois exemples démontrent la méthode, pas la performance générale.

À montrer / à faire

Lis les quatre scores avec leur sens, puis donne la limite du petit échantillon. En discussion, ouvre les exemples RAGAS du rapport.

▼ Comprendre simplement

Fidélité : est-ce écrit dans les fiches ? Pertinence : est-ce que ça répond à la question ? Précision du contexte : les passages utiles arrivent-ils bien placés ? Correction : est-ce conforme à ce que la réponse humaine attend ? Une réponse peut être fidèle à un document, mais incomplète ou mal ciblée.

▼ Si le jury creuse la technique

RAGAS 0.4.3 ; juge ministral-8b-2512 ; embeddings mistral-embed. Faithfulness juge les affirmations soutenues. ContextPrecision s'appuie sur une référence pour évaluer les contextes récupérés et leur classement : 1,0 ne signifie ni tous les chunks utiles ni rappel parfait. AnswerCorrectness combine comparaison factuelle et similarité. Le script appelle les métriques séquentiellement avec reprises après erreurs transitoires. Les seuils fort $\geq 0,8$ et acceptable $\geq 0,6$ sont des conventions du projet. Le même fournisseur sert à générer et juger : biais possible, review humaine nécessaire. Les références courtes peuvent pénaliser des développements utiles, mais cela ne justifie pas les faits inventés.

▼ La preuve dans le projet

scripts/evaluate_ragas.py ; src/openagenda_rag/ragas_evaluation.py ; outputs/evaluation/ragas_results.json ; rapport, section 7.

Le piège à éviter

0,900 n'est pas un taux de satisfaction de 90 %. 1,000 n'est pas un rappel de 100 %. Ces scores historiques n'ont pas été recalculés aujourd'hui.

Transition à dire : « Ces résultats et la démonstration permettent d'identifier les limites prioritaires. »

Les limites définissent le travail avant production

L'idée à retenir : Actualiser les données, renforcer le refus et mesurer davantage.

La première limite est le corpus : il est limité à Paris et figé en juillet. Pour recommander des sorties actuelles, il faut automatiser son actualisation et filtrer les dates au moment de la demande. La deuxième concerne la recherche : le top-k renvoie les voisins les plus proches même lorsqu'ils ne répondent pas réellement à la question. Le refus repose alors sur le prompt et le modèle. Un seuil calibré et des filtres explicites seraient plus robustes. La troisième limite est l'exploitation. L'index est local, mais les embeddings de question et la génération appellent Mistral. Aujourd'hui, Mistral Small était limité. La démo a été rétablie avec Ministral 8B, sans changer les embeddings. Les erreurs fournisseur sont désormais contrôlées. Ce nouveau modèle doit être réévalué ; les anciens scores ne lui sont pas attribués. Enfin, trois cas RAGAS restent insuffisants pour conclure à une qualité générale.

À montrer / à faire

Relie chaque limite à une amélioration. Si la démo enregistrée a été utilisée, rappelle simplement la cause observée.

▼ Comprendre simplement

Tu n'as pas besoin de défendre le prototype comme parfait. Tu dois connaître ses limites, montrer leur effet et proposer une action concrète. La panne initiale a été contournée par un modèle Mistral disponible. Ce changement permet la démo, mais ne prouve pas une qualité équivalente.

▼ Si le jury creuse la technique

Pas de reranker, recherche hybride, seuil de distance, filtre de métadonnées par question, authentification globale, limitation de débit ni supervision livrés. Reconstruction destructive sur le répertoire cible : pas de bascule atomique et risque de concurrence à traiter en production. Pickle de FAISS chargé avec `allow_dangerous_deserialization=True` : uniquement artefacts de confiance. Les noms latest et les dépendances transitives non totalement figées limitent la reproductibilité stricte.

▼ La preuve dans le projet

README.md : limites ; src/openagenda_rag/rag.py ; api.py ; indexing.py ; état de préparation.

Le piège à éviter

Ne prétends pas que Docker rend le projet entièrement hors ligne, qu'un seuil seul comprend toutes les contraintes, ou que l'API est prête pour une exposition publique.

Transition à dire : « Je propose donc une suite mesurée, fondée sur les risques identifiés. »

Un pilote mesuré est la prochaine étape

L'idée à retenir : Le POC livre la chaîne complète ; la valeur produit reste à mesurer.

Pour conclure, le POC livre une chaîne complète : des données réelles, un index persistant, une génération augmentée, une API, Docker, des tests et une méthode d'évaluation. Je recommande d'abord d'actualiser le corpus et d'ajouter les garde-fous de ville et de date. Ensuite, d'élargir le jeu annoté et de comparer les paramètres de recherche. Un pilote doit mesurer la réponse utile, les refus corrects, les clics vers les sources, la latence et le coût. Je vous montre brièvement les preuves dans le rapport et le dépôt. Le rapport documente les choix et les résultats. Dans le dépôt, scripts permet de relancer les étapes, src contient le moteur et l'API, tests les validations, et seed-data le corpus de référence. Ces éléments permettent à une autre personne de reproduire et d'examiner le POC. Je suis prêt à répondre à vos questions.

À montrer / à faire

30 s de conclusion ; 20 s rapport (architecture, évaluation, limites) ; 25 s dépôt (scripts/, src/, tests/, seed-data/).
Retourne ensuite au deck. Tous les liens sont dans le guide.

▼ Comprendre simplement

La conclusion répond à la demande du client : est-ce faisable, quelles preuves avons-nous, que faut-il faire ensuite ? Tu proposes une expérimentation, pas une promesse de conversion ou de qualité universelle.

▼ Si le jury creuse la technique

Priorités : fraîcheur et contraintes métier ; jeu annoté diversifié et cas négatifs ; optimisation mesurée ; gestion quotas/erreurs/secrets ; authentification, logs, CI, tests de charge et mises à jour atomiques. Les clics et la satisfaction doivent être observés avec des utilisateurs. Aucun coût précis ni objectif de latence de production n'a été mesuré dans ce POC.

▼ La preuve dans le projet

README.md ; outputs/rapport-technique-openagenda-rag.pdf ; <https://github.com/hgbe-GH/Projet7>.

Le piège à éviter

Ne consacre pas plusieurs minutes à parcourir le dépôt. La présentation doit rester entre 10 et 20 minutes, cible 15.

Transition à dire : « Discussion : écoute la question, réponds en une phrase, puis donne la preuve ou la limite utile. »

La démo, geste par geste

Tu as **2 min 20** pour la démo et son interprétation. [Ouvrir le démonstrateur](#) · [Swagger](#).

1. **20 s** : montre /health et les routes Swagger. Dis « Le serveur et le contrat sont disponibles ; health ne prouve pas à lui seul que Mistral répond. »
2. **50 s** : sélectionne Concert Fishers. Si le mode direct fonctionne, appelle /ask. Sinon, annonce l'échec et clique explicitement « Réponse enregistrée ». Montre date, lieu, URL et premier événement source.
3. **40 s** : sélectionne Lyon. Montre le refus enregistré ou reçu en direct et la ville Paris dans les sources. Explique que ces sources sont récupérées, sans être des recommandations adaptées à Lyon.
4. **30 s** : dis la limite : pas de seuil ni filtre ville dans le retrieve. Retourne au deck, slide 9.

Si Mistral reste limité aujourd'hui

« L'API locale est disponible, mais le fournisseur Mistral renvoie actuellement une limite d'usage. Je vous montre le contrat et les résultats d'une répétition réelle enregistrée, que je distingue d'un appel live. Le résultat permet d'examiner le fonctionnement, mais l'indisponibilité externe empêche la démonstration de bout en bout en direct dans cette session. »

Dans la page de démonstration, aucun échec direct n'est remplacé automatiquement par un succès enregistré. La date et la provenance sont affichées. Les enregistrements existent localement et restent consultables sans internet.

Alternative par terminal

```
curl -s http://127.0.0.1:8000/health

curl -s -X POST http://127.0.0.1:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "Parle-moi de Concert Fishers a Paris"}'
```

Ne montre pas un détail d'erreur brut. Ne lance pas /rebuild, une collecte complète ou RAGAS pendant la soutenance. Le script demo_api_5min.py s'arrête désormais au premier échec ; la page permet aussi de contrôler une tentative à la fois.

Ce qu'il faut lire dans le JSON

Champ	Interprétation
question	Demande reçue.
answer	Texte généré par le modèle, ou refus.
sources	Événements récupérés, dédoublés par UID ; pas la preuve que tous sont cités.

Champ	Interprétation
retrieved_chunk_count	Nombre de passages fournis au modèle, distinct du nombre de sources uniques. Ce champ n'était pas conservé dans le rapport de démo historique et n'est pas inventé dans sa relecture.

Le rapport et le dépôt, sans te perdre

[Ouvre le rapport technique](#). Montre l'architecture et la section 7 d'évaluation, puis dis : « Le rapport relie les choix, les résultats et les limites. »

[Ouvre le dépôt GitHub](#). Montre `scripts/`, `src/openagenda_rag/`, `tests/` et `seed-data/`, puis reviens au deck. Le README suffit comme rapport demandé par le brief ; le PDF complète la livraison.

Si le jury demande « montre le code principal », ouvre `rag.py` et `answer_question()` : récupération, contexte, génération, sources. Pour l'architecture : `service.py`. Pour le découpage : `indexing.py`. Pour les routes : `api.py`.

Les réponses aux questions de JérémY

Écoute la question jusqu'au bout. Donne d'abord la réponse courte, puis développe si nécessaire. Quand tu ne sais pas, distingue ce qui a été vérifié de ce qui demande une mesure.

▼ Pourquoi le RAG, et avez-vous entraîné le modèle ?

Le RAG ajoute les passages du catalogue au moment de la question. Je n'ai ni entraîné ni ajusté Mistral ; j'ai construit un index et un prompt.

Les données peuvent évoluer sans réentraîner le modèle : on recollecte puis on réindexe. Le RAG ne supprime pas les erreurs de génération.

Preuve : rag.py : answer_question()

▼ Pourquoi Mistral et ces deux modèles ?

Mistral est demandé par le brief. mistral-embed sert à rechercher. Le POC évalué utilisait mistral-small-latest ; la démo utilise ministral-8b-2512 car Small était limité aujourd'hui.

Le modèle alternatif a réussi les deux scénarios API réels. Ce n'est pas un benchmark comparatif et les scores RAGAS de juillet restent ceux de Small. Comme les embeddings n'ont pas changé, aucun réindexage n'est nécessaire. Le nom de version est fixé pour cette démo.

Preuve : settings.py ; rag.py ; indexing.py

▼ Pourquoi FAISS et quel index utilisez-vous ?

FAISS permet une recherche vectorielle locale sans serveur supplémentaire. L'index livré est IndexFlatL2, une recherche exacte par distance euclidienne au carré.

Il contient 14 903 vecteurs de 1 024 dimensions. Les dimensions ne sont pas des catégories nommées. Pour beaucoup plus de données, il faudrait mesurer les coûts de mémoire et de recherche avant de choisir un index approximatif ou une base serveur. Le code n'applique pas de normalisation explicite pour une recherche cosinus.

Preuve : seed-data/index/faiss/index.faiss ; indexing.py

▼ Pourquoi 1 000 caractères, 200 de chevauchement et top-k 4 ?

Ce sont les réglages de départ du POC : des passages assez courts, un chevauchement pour les frontières et quatre contextes pour limiter le

bruit.

Ils n'ont pas été prouvés optimaux. Je comparerais plusieurs tailles et valeurs de k en mesurant le rappel des événements attendus, la fidélité, la latence et le coût. RecursiveCharacterTextSplitter respecte les séparateurs ; ce n'est pas une découpe fixe tous les 800 caractères.

Preuve : index_manifest.json ; settings.py

▼ Vos événements sont-ils encore récents et à venir ?

Le seed couvre la première date du 10 juillet 2025 au 10 juillet 2026. C'était le périmètre construit en juillet ; il n'est pas maintenu à jour au 1er octobre.

Certains événements ont maintenant plus d'un an, et la borne haute exclut les nouveaux événements qui commencent après juillet. Quelques événements récurrents se prolongent après la borne. Pour remplir durablement le besoin d'événements à venir, il faut une collecte planifiée avec un horizon futur et un filtrage des occurrences au moment de la réponse. Je reconnais cet écart actuel.

Preuve : fetch_manifest.json ; ingestion.py : build_records_where_clause()

▼ Pourquoi Opendatasoft plutôt que l'API native OpenAgenda ?

La mission renvoie au dataset public OpenAgenda d'Opendatasoft ; c'est cette source qui est utilisée et documentée.

La collecte est sans clé sur cette source. Elle normalise un schéma qui pourrait être alimenté par une autre source. Je ne prétends pas utiliser les endpoints natifs OpenAgenda.

Preuve : ingestion.py : OpenAgendaClient

▼ Comment évitez-vous les hallucinations ?

Le prompt impose le contexte et le refus si l'information manque ; les sources permettent un contrôle. Cela réduit le risque sans garantir zéro erreur.

Je compléteraï par des filtres explicites, un seuil calibré, un contrôle des faits, des tests adverses et une revue humaine. Les descriptions externes doivent aussi être traitées comme des données non fiables pour éviter les injections de prompt.

Preuve : rag.py : SYSTEM_PROMPT

▼ Pourquoi Lyon retourne-t-il quand même des sources ?

Le top-k renvoie les voisins les plus proches, même si leur ville ne correspond pas. Ce sont des documents récupérés, pas forcément des

recommandations.

Le refus Lyon observé vient du modèle lisant le contexte Paris. Il n'existe pas de blocage géographique codé ni de seuil. Un filtre ville ou un rejet mesuré rendrait le comportement plus fiable et pourrait éviter un appel de génération inutile.

Preuve : rag.py : build_retriever(), build_source_entries()

▼ Que signifient 0,900 et 1,000 ?

0,900 mesure la fidélité au contexte sur trois cas. 1,000 mesure la précision et le classement des contextes par RAGAS, pas la récupération de tous les événements pertinents.

Ce ne sont ni une satisfaction de 90 % ni une exactitude universelle de 100 %. Il manque une mesure explicite de rappel du retrieval et un échantillon plus large. Les résultats datent du 23 juillet et peuvent varier avec le juge.

Preuve : ragas_results.json ; ragas_evaluation.py

▼ Pourquoi la correction n'est-elle que 0,683 ?

Une réponse peut être soutenue par un contexte mais s'écarter de la référence, oublier une information ou ajouter un détail fragile.

Je relis les cas plutôt que de déduire une cause unique du score. Le cas famille obtient environ 0,581 en correction : des horaires sont généralisés à plusieurs visites. Les références doivent être enrichies lorsque nécessaire, et les réponses doivent mieux distinguer les faits sourcés des conseils.

Preuve : rapport, section 7 ; ragas_results.json

▼ Quatre cas corrects : votre précision est-elle de 100 % ?

Non. Ce classement porte sur quatre exemples et sur une règle locale qui accepte un titre présent dans la réponse ou dans les sources.

La similarité locale est lexicale avec SequenceMatcher, pas sémantique. L'exact match est nul dans la sortie enregistrée malgré quatre classements corrects. Cette évaluation vérifie surtout des repères ; elle doit être complétée par les faits, les dates, le refus et des annotations plus nombreuses.

Preuve : evaluation.py : classify_result(), match_expected_titles()

▼ Pourquoi seulement trois exemples RAGAS ?

Ils démontrent une évaluation réellement exécutée avec les contextes exacts. Ils sont insuffisants pour conclure statistiquement.

J'ajouterais villes, catégories, dates, questions ambiguës, négations, hors corpus et paraphrases. Je séparerais les cas utilisés pour choisir les réglages de ceux utilisés pour les évaluer et j'intégrerais une revue humaine indépendante du juge.

Preuve : tests/fixtures/ragas_eval_dataset.csv

▼ Les 76 tests appellent-ils réellement Mistral ?

La suite de tests utilise des doublures pour vérifier le code sans dépendre du réseau. La démo API et l'évaluation enregistrée font les appels réels séparément.

Les tests ne prouvent ni la disponibilité du fournisseur ni la qualité de toutes les réponses. Les appels réels ont été vérifiés séparément aujourd'hui. Dix-sept régressions supplémentaires couvrent les erreurs fournisseur, l'arrêt des répétitions et la gestion des dates dans le contexte.

Preuve : tests/conftest.py ; test_rag.py ; test_api.py

▼ Est-ce réellement hors ligne ?

Non. Le corpus et FAISS sont locaux, mais Mistral est appelé pour l'embedding de la question et la génération.

Les pages et les résultats enregistrés fonctionnent hors ligne ; ils ne remplacent pas un RAG live. Une version entièrement locale demanderait des modèles locaux compatibles et un nouvel index si l'embedding change.

Preuve : rag.py ; indexing.py : _build_embeddings()

▼ Que fait /rebuild exactement ?

Il relit le Parquet, redécoupe, recalcule les embeddings et remplace l'index. Il invalide ensuite le retriever et le client de chat.

Il ne télécharge pas de nouveaux événements. Pour actualiser les données il faut d'abord fetch_events.py, puis l'indexation. Cela peut coûter et durer ; je ne le lance pas pendant la soutenance. En production, une reconstruction dans un dossier séparé puis une bascule atomique éviterait les conflits.

Preuve : service.py : rebuild() ; indexing.py : rebuild_index_artifacts()

▼ Comment protégez-vous la clé et les endpoints ?

La clé est dans l'environnement ou .env ignoré par Git. Docker ne la copie pas. /rebuild dispose d'un jeton optionnel.

Le jeton n'est pas obligatoire dans le POC ; avant exposition il faut l'activer, ajouter authentification et limites d'appels. Les erreurs fournisseur sont déjà filtrées. L'index pickle doit provenir d'une source de confiance. Je montre .env.example, jamais .env.

Preuve : .gitignore ; .dockerignore ; api.py

▼ Que garantit Docker et /health ?

Docker fournit le code, les dépendances et un seed dans un environnement commun. /health confirme que le serveur répond et expose sa configuration.

Health ne prouve pas que la clé fonctionne ni que l'index est chargé. Le chargement est paresseux à la première question. Le volume conserve l'index de travail. Une readiness complète devrait vérifier le corpus et un contrôle adapté du fournisseur, sans appels coûteux à chaque sondage.

Preuve : Dockerfile ; docker_entrypoint.sh ; service.py : health()

▼ Comment avez-vous débloqué Mistral ?

J'ai isolé les appels : les embeddings fonctionnaient, la génération avec Small renvoyait 429. Ministral 8B répondait ; je l'ai configuré puis vérifié les deux scénarios RAG.

La clé était valide et identique dans Docker. L'accès aux limites administratives n'était pas autorisé avec cette clé : la limite précise n'est pas connue. L'API traite désormais les erreurs fournisseur sans exposer leurs détails ; le script de démo ne relance plus 60 fois les appels. Il faudra réévaluer le modèle alternatif et surveiller les limites par modèle.

Preuve : Vérification du 1er octobre ; api.py : gestion des exceptions

▼ Avez-vous une CI/CD et mesuré la montée en charge ?

Non. Les scripts et les tests sont livrés ; aucune pipeline CI/CD ni benchmark de charge n'est implémenté.

Je proposerais tests sans secret à chaque push, construction Docker, contrôles de disponibilité, puis déploiement contrôlé. Je mesurerais p50/p95, débit, taux d'erreur, coûts et ressources sur une charge représentative. Une démo courte ne permet pas de les extrapoler.

Preuve : tests/ ; Dockerfile ; README.md

▼ Quel est le coût et la valeur pour Puls-Events ?

La valeur attendue est une découverte plus naturelle avec des sources vérifiables. Elle n'a pas encore été mesurée auprès d'utilisateurs.

Coût = embeddings de construction et mise à jour + embedding par question + tokens de génération, et éventuellement juge RAGAS. Je ne donne pas de prix non vérifié. Un pilote comparerait satisfaction, clic vers source, refus correct, latence et coût par demande à une recherche classique.

▼ Comment comparer un nouveau modèle d'embedding ?

Je reconstruis l'index avec ce modèle et j'utilise le même modèle pour les questions. On ne peut pas mélanger deux espaces vectoriels.

Même si les deux vecteurs ont la même dimension, leurs coordonnées ne sont pas forcément comparables. Je garde le même corpus et jeu d'évaluation pour comparer rappel, pertinence, coût et latence. Le manifeste trace le modèle utilisé.

Preuve : index_manifest.json ; service.py ; settings.py

▼ Comment rendre le système plus utile pour des dates et lieux précis ?

J'ajouterais des contraintes métier explicites sur la ville et les occurrences, en complément de la recherche sémantique.

Un événement récurrent ne doit pas être interprété comme une disponibilité continue entre first_timing et last_timing. Le corpus brut conserve timings ; le POC utilise surtout première et dernière dates. Un futur moteur doit sélectionner l'occurrence qui correspond à la demande et vérifier accessibilité, tarifs et public lorsque les données existent.

Preuve : ingestion.py : _extract_timings() ; métadonnées de FAISS

Répondre sans inventer et garder le fil

Si tu oublies un terme

Reviens à quatre questions : **qu'est-ce qui entre, que fait cette brique, qu'est-ce qui sort, pourquoi ce choix ?** Pour FAISS : vecteur de question → comparaison des distances → passages proches → recherche locale simple.

Si le jury te pose une question dont tu ignores la réponse

« Sur ce point, je préfère être précis : ce que j'ai vérifié dans le POC est... Pour le détail que vous demandez, je dois consulter le code ou mesurer avant de conclure. »

Si une erreur est pointée

Reconnais l'écart, explique son effet, donne la correction. Exemple : « Vous avez raison, une fenêtre figée en juillet n'est plus un catalogue à jour en octobre. Il faut une collecte planifiée et des filtres d'occurrences ; le mécanisme RAG peut être conservé. »

Si tu prends du retard

Utilise « L'idée à retenir » sur les fiches suivantes. Garde impérativement les données, les modèles, la démo, les résultats et les limites. Réduis le parcours du dépôt et les explications secondaires. Si tu es trop rapide, explique un exemple concret de chunk ou un score ; évite de meubler.

Si Docker ne répond plus

```
docker compose ps
docker compose up -d --no-build
```

Une seule relance. Si elle échoue, montre l'enregistrement et l'état réel ; ne passe pas plusieurs minutes à déboguer devant le jury.

Si le support HTML ne s'affiche plus

Utilise la présentation PDF ou PowerPoint synchronisée. Le guide peut aussi être imprimé. Tous les textes sont autonomes ; seuls les appels Mistral et GitHub nécessitent internet.

Ce que tu peux dire pendant le débrief

Quand l'évaluateur quitte le rôle de Jérémie, écoute son retour. Tu peux dire : « Le projet m'a appris à distinguer la qualité des données, la recherche et la fidélité de génération. La suite prioritaire est d'actualiser les données et d'élargir l'évaluation. » Adapte cette phrase à ce que tu as réellement compris et vécu.

MISSION ET PIÈCES LIÉES

Les exigences et les preuves disponibles

Cette préparation reprend la mission, ses six étapes, les consignes de soutenance et les dix rubriques du modèle de rapport fourni. Le document exporté contient aussi le cours RAG avec un cas municipal : ces exercices pédagogiques ne sont pas des livrables de la mission Puls-Events.

Exigence	Preuve	État / limite
RAG LangChain, Mistral, FAISS	rag.py, indexing.py, service.py	Livré ; génération distante.
Scripts de reconstruction	fetch_events.py, build_index.py ; /rebuild	Livrés ; rebuild relit le Parquet, sans collecte.
Zone et événements récents	Manifeste Paris, juillet 2025 → juillet 2026	Corpus figé ; certains événements dépassent un an aujourd’hui. Actualisation nécessaire.
API REST exploitable	/ask, /rebuild, /health, /docs	Serveur disponible ; /ask bloqué actuellement par Mistral.
Rapport technique	README + PDF existant	Couvre les choix, résultats, limites et perspectives ; PDF daté du 28 juillet.
Tests et jeu annoté	76 tests ; 4 cas locaux ; 3 cas RAGAS	Tests passent aujourd’hui ; petit échantillon métier.
Docker	Dockerfile, Compose, seed et volume	Image existante démarrée ; aucun nouveau build complet affirmé.
Démo live	Swagger et démonstrateur direct	Deux scénarios réussis en direct avec Ministral 8B ; enregistrement du jour disponible en secours.
Présentation 10 à 15 slides	12 slides HTML et export synchronisé	HTML pour présenter, formats de secours pour le brief.
Parcours rapport / dépôt	Slide 12 ; guide démo	Inclus dans les 15 minutes.
ZIP et lien GitHub	Archive historique dans outputs ; dépôt hgbe-GH/Projet7	La nouvelle archive de soutenance rassemble les supports actuels ; code, preuves et supports inclus dans la mise à jour GitHub ; dépôt plateforme à effectuer.

Le modèle de rapport et la fiche d’autoévaluation

Objectifs → architecture → préparation/vectorisation → modèle NLP → base vectorielle → API → évaluation → perspectives → dépôt → annexes. Le PDF existant suit ce modèle. La fiche d’autoévaluation liée à la mission a également été consultée ; elle demande notamment un schéma UML clair, des tests relançables, des résultats documentés et une API accessible. Le complément ci-dessous fournit le schéma des composants et distingue les éléments vérifiés des points à compléter.

[Architecture détaillée et autoévaluation ↗](#)

Traçabilité des affirmations

Volumes et configuration : manifestes versionnés et inspection du seed. Comportement : lecture de rag.py, indexing.py, service.py et api.py. Tests : exécution du 1er octobre. Scores : ragas_results.json, généré le 23 juillet 2026 à 14:36:49 UTC. Démo du jour : demo_api_2026-10-01.json, deux scénarios, modèle, horodatage et durée globale conservés. Les scores ne sont ni extrapolés ni remplacés par des résultats inventés.

Guide et diaporama synchronisés sur les mêmes 12 étapes. Les supports précédents sont conservés dans archives/.